

Spesifikasi Teknis: Mesin Pencarian Strategi Evolusioner (ESSE)

Arsitektur Sistem High-Performance (Rust Ecosystem)

Sistem ini merupakan Mesin Pencarian Strategi Evolusioner yang beroperasi secara otonom untuk mengekstraksi *alpha* dari data pasar beresolusi tinggi. Arsitektur sistem dibangun di atas ekosistem **Rust** untuk mencapai latensi ultra-rendah, keamanan memori yang ketat, dan konkurensi masif yang dijamin oleh *borrow checker*. Sistem ini didedikasikan secara murni sebagai mesin riset dan penemuan strategi (*Strategy Discovery Engine*).

1. Data Layer

Sistem menggunakan arsitektur *Decoupled Storage* untuk memisahkan beban baca data mentah (Tick/Candle) dan penulisan hasil komputasi (Metrik/DNA).

A. Market Data Store (ClickHouse)

Bertindak murni sebagai penyedia data mentah berkecepatan tinggi. Desain skema dioptimalkan menggunakan Enum8 dan kompresi ZSTD.

1. Tabel ticks (Resolusi Mikro - Event-driven Backtest):

```
CREATE TABLE geonera.ticks (  
  `timestamp` DateTime64(6, 'UTC') CODEC(Delta(8), ZSTD(3)),  
  `instrument` Enum8('XAUUSD'=1, 'AUDJPY'=2, 'CHFJPY'=3, 'CADJPY'=4, 'GBPJPY'=5,  
  'EURJPY'=6, 'NZDJPY'=7, 'USDJPY'=8, 'GBPUSD'=9, 'EURUSD'=10, 'USDCHF'=11, 'USDCAD'=12,  
  'AUDUSD'=13, 'NZDUSD'=14, 'EURGBP'=15, 'EURNZD'=16) CODEC(ZSTD(3)),  
  `bid` Decimal(9, 5) CODEC(Delta(8), ZSTD(1)),  
  `ask` Decimal(9, 5) CODEC(Delta(8), ZSTD(1)),  
  `bid_volume` UInt32 DEFAULT 0 CODEC(Delta(4), ZSTD(1)),  
  `ask_volume` UInt32 DEFAULT 0 CODEC(Delta(4), ZSTD(1)),  
  `version` DateTime64(3, 'UTC') CODEC(Delta(8), ZSTD(3))  
) ENGINE = ReplacingMergeTree(version)  
PARTITION BY toYYYYMM(timestamp)  
PRIMARY KEY (instrument, timestamp)  
ORDER BY (instrument, timestamp)  
SETTINGS index_granularity = 8192;
```

2. Tabel candles (Resolusi Agregat - Analisis & Regime):

```
CREATE TABLE geonera.candles (  
  `timestamp` DateTime64(6, 'UTC') CODEC(Delta(8), ZSTD(3)),  
  `instrument` Enum8('XAUUSD'=1, 'AUDJPY'=2, 'CHFJPY'=3, 'CADJPY'=4, 'GBPJPY'=5,  
  'EURJPY'=6, 'NZDJPY'=7, 'USDJPY'=8, 'GBPUSD'=9, 'EURUSD'=10, 'USDCHF'=11, 'USDCAD'=12,  
  'AUDUSD'=13, 'NZDUSD'=14, 'EURGBP'=15, 'EURNZD'=16) CODEC(ZSTD(3)),  
  `open` Decimal(9, 5) CODEC(Delta(8), ZSTD(1)),  
  `high` Decimal(9, 5) CODEC(Delta(8), ZSTD(1)),  
  `low` Decimal(9, 5) CODEC(Delta(8), ZSTD(1)),  
  `close` Decimal(9, 5) CODEC(Delta(8), ZSTD(1)),  
  `volume` UInt32 DEFAULT 0 CODEC(Delta(4), ZSTD(1)),  
  `version` DateTime64(3, 'UTC') CODEC(Delta(8), ZSTD(3))  
) ENGINE = ReplacingMergeTree(version)  
PARTITION BY toYYYYMM(timestamp)  
PRIMARY KEY (instrument, timestamp)  
ORDER BY (instrument, timestamp)  
SETTINGS index_granularity = 8192;
```

```

`timestamp` DateTime('UTC') CODEC(Delta(8), ZSTD(3)),
`instrument` Enum8(...) CODEC(ZSTD(3)), -- Enum 16 Instrumen
`timeframe` Enum8('m1'=1, 'm2'=2, 'm3'=3, 'm4'=4, 'm5'=5, 'm6'=6, 'm10'=7, 'm12'=8, 'm15'=9,
'm20'=10, 'm30'=11, 'h1'=12, 'h2'=13, 'h3'=14, 'h4'=15, 'h6'=16, 'h8'=17, 'h12'=18, 'd1'=19, 'w1'=20)
CODEC(ZSTD(3)),
`open` Decimal(9, 5) CODEC(Delta(8), ZSTD(3)),
`high` Decimal(9, 5) CODEC(Delta(8), ZSTD(3)),
`low` Decimal(9, 5) CODEC(Delta(8), ZSTD(3)),
`close` Decimal(9, 5) CODEC(Delta(8), ZSTD(3)),
`tick_count` UInt32 CODEC(T64, ZSTD(3)),
`min_spread` Decimal(9, 5) CODEC(Delta(8), ZSTD(3)),
`max_spread` Decimal(9, 5) CODEC(Delta(8), ZSTD(3)),
`avg_spread` Decimal(9, 5) CODEC(Delta(8), ZSTD(3)),
`total_bid_volume` UInt64 CODEC(T64, ZSTD(3)),
`total_ask_volume` UInt64 CODEC(T64, ZSTD(3)),
`vwap` Decimal(9, 5) CODEC(Delta(8), ZSTD(3)),
`version` DateTime64(3, 'UTC') CODEC(Delta(8), ZSTD(3))
) ENGINE = ReplacingMergeTree(version)
PARTITION BY (instrument, toYYYYMM(timestamp))
PRIMARY KEY (instrument, timeframe, timestamp)
ORDER BY (instrument, timeframe, timestamp)
SETTINGS index_granularity = 8192;

```

B. Research Database (PostgreSQL)

Menggunakan sqlx untuk kueri *Type-safe*.

1. **strategy_dna**: Blueprint (AST Enum, Params), status validasi, metrik IS/OOS.
2. **trial_logs**: Log metrik per *Window ID* dan *Regime ID*.
3. **trade_logs**: Setiap mutasi eksekusi raw (*pips*, *net_pips*, *Jendela In-Sample / Out-of-Sample*).
4. **monthly_metrics**: Agregasi performa per bulan.

2. GP + Bayesian Engine

Sistem mengoptimasi logika dan parameter secara konvergen.

- **A. Penemuan Struktural (Genetic Programming - GP)**: Representasi *Abstract Syntax Tree* (AST) dengan **Enums dan Pattern Matching** Rust.
 - **Primitive Set**: add, sub, mul, div, max2, min2, neg, square, cube, log, sqrt, sigmoid, sign.
 - **Bloat Control**: Pohon dibatasi minimal 2 *node*, maksimum 6 tingkat kedalaman, dan **maksimal 60 node total**.
- **B. Optimasi Numerik (Bayesian FFI - PyO3)**: Karena TPE berstandar industri ada di

Optuna (Python), **Rust memanggil pustaka Optuna melalui FFI (PyO3)**. Rust mengirimkan batas sampel, Optuna mengembalikan tebakan numerik, dan Rust mengevaluasinya secara *native*.

- `sl_pips`: 10 hingga 50 pips.
- `tp_pips`: **Wajib $\geq 1.5 \times \text{SL}$** , hingga 150 pips.
- `signal_threshold`: 0.05 hingga 0.95.
- Fitur *Windows*: 3 - 300 *tick*.
- **C. Diversity Engine (Anti-Stagnation)**: Mencegah konvergensi prematur ("inses" genetik) di mana populasi menjadi terlalu homogen.
 - **Genotypic Distance**: Mengukur jarak kemiripan struktur pohon AST antar-individu menggunakan *Tree Edit Distance*.
 - **Phenotypic Distance**: Jika strategi baru memiliki kemiripan *trade* > 85% dengan *Elite* yang sudah ada, sistem akan memaksa mutasi destruktif atau memberikan penalti *Crowding Penalty*.

3. Feature Microstructure

Menghindari indikator klasik (MACD/RSI), AST dipaksa mengeksploitasi 9 fitur mikrostruktur pasar:

1. **Order Book Imbalance (OBI)**: Tekanan volume bid vs ask.
2. **Tick Velocity**: Rata-rata kecepatan perubahan *mid-price*.
3. **Spread Dynamics**: Volatilitas melebarnya *spread*.
4. **Tick Density**: Konsentrasi aktivitas (jumlah *tick* per detik).
5. **Volume Clock Skew**: Ketimpangan aktivitas volume bid/ask.
6. **Mid Price Momentum**: Selisih momentum jendela pendek vs panjang.
7. **Rolling Skewness & Kurtosis**: Mengukur probabilitas ekor (*fat tails*).
8. **Volume Weighted Spread**: Bentang harga riil berbobot likuiditas.
9. **Rolling Hurst Exponent**: Proksi kuantitatif pengukur sifat fraktal data *tick*.
Mengidentifikasi apakah sinyal mikro saat ini berada pada dinamika *mean-reverting* ($H < 0.5$) atau *trending* ($H > 0.5$).

4. Backtest Engine (Rust)

Engine simulasi tingkat *tick* berkecepatan vektor untuk menguji populasi strategi.

- **A. Gapless Chunking & Streaming Processing (Anti-OOM)**: Mencegah *Crash* memori saat mengevaluasi terabytes data *tick*. Data ditarik dari ClickHouse secara *streaming* per blok (misal: per bulan), namun dirancang **tanpa celah waktu (gapless)** melalui mekanisme:
 - **Overlapping Warm-up Windows**: Kueri ClickHouse selalu menyertakan N baris ekstra dari masa lalu sebelum batas awal *chunk* baru. Ini memastikan fitur *rolling window* langsung terisi dan akurat sejak detik pertama *chunk* tersebut.
 - **Stateful Context Hand-off**: Status internal strategi dipertahankan dalam satu objek

Context di memori Rust dan "dioper" secara mulus dari *chunk* 1 ke *chunk* 2.

- **B. Hybrid CPU/GPU AST Evaluation:** Jika menggunakan CPU: Evaluasi Enums dieksekusi menggunakan paralelisasi data rayon di seluruh core.
 - **Akselerasi GPU:** Mendelegasikan komputasi ke VRAM via cudarc / wgpu. Pohon Genetik (AST) diterjemahkan secara *Just-in-Time* menjadi instruksi PTX/SPIR-V.
- **C. Execution Guardrails & Transaction Cost:**
 - **Biaya Transaksi Absolut:** Setiap transaksi wajib dikurangi *Base Spread* dan komisi pialang untuk mendapatkan kalkulasi **Net Pips**.
 - **Stateful Signal Transition:** Entri dieksekusi hanya pada transisi HOLD -> BUY.
 - **Max Holding Time:** Penahanan kedaluwarsa jika lewat batas waktu menggunakan "Waktu Pasar Aktif".
- **D. Noise Injection Testing:** Menguji klon strategi pada data yang dikotori artifisial (menyuntikkan *delay* eksekusi 50ms-200ms dan melebarkan *spread* historis secara acak). Strategi yang runtuh oleh *noise* dibuang.
- **E. Hard Filters (Eliminasi Instan):**
 - Frekuensi trade < 10 atau > 20 per bulan.
 - Risiko < 10 atau > 50 pips.
 - Drawdown > 500 pips sebulan.
 - Tidak ada loss (anomali Undefined Profit Factor).

5. Walk-Forward Validation

Untuk membunuh fenomena "Alpha Palsu" (Curve Fitting), data historis wajib dipecah:

- **Multi-Window Rolling Split:** Sistem melatih (In-Sample) dan menguji (Out-of-Sample) dalam matriks jendela bergeser.
Contoh: Latih di bulan 1-6 (IS), Uji di bulan 7-8 (OOS). Bergeser: Latih di bulan 3-8 (IS), Uji di bulan 9-10 (OOS). Strategi diwajibkan *survive* di banyak irisan *timeline* maju.
- **Pre-Tuning IS Blinding:** Proses Bayesian Optimization hanya boleh mengonsumsi data blok In-Sample.

6. Regime Classification

Setiap "Jendela" tidak dinilai sama rata. Mesin secara otomatis mendeteksi profil makro menggunakan **Proksi Matematika Kuantitatif**:

- **Kuantifikasi Rezim: Fractal/Trend Detection (Hurst Exponent):** Jika $H > 0.55$, pasar dideklarasikan sebagai **Trending**. Jika $H < 0.45$, pasar dideklarasikan sebagai **Ranging/Mean-Reverting**. Jika di sekitar 0.5 , pasar dianggap *Random Noise*.
 - **Volatility Detection:** Menggunakan **ATR (Average True Range)** atau *Rolling Variance* untuk mendikotomi rezim *High Volatility* vs *Low Volatility*.
- Berdasarkan metrik proksi di atas, setiap *Window* OOS dilabeli atribut makro:
 - HIGH_VOL_TRENDING | LOW_VOL_RANGING | HIGH_VOL_RANGING |

7. Degradation Filtering

Mengeksekusi hukuman keras bagi strategi penghafal data (*overfitted*):

- **OOS Degradation Rule:** Strategi otomatis dibunuh (`OOS_DEGRADATION_FAILED`) jika Profit Factor OOS-nya hancur/jatuh melampaui toleransi minimum (< 1.5) di 80% total Jendela OOS.
- **Regime Survival:** Boleh berkinerja netral (*flat*) pada rezim pasar non-target, tetapi dilarang hancur (*drawdown* fatal) saat rezim berubah haluan.
- **Incubation State (Forward Testing):** DNA yang lulus saringan ketat ini akan masuk fase inkubasi (validasi data riil masa depan ter- *update* tanpa risiko eksekusi).

8. Fitness Scoring (OOS & Risk Heavy)

Fungsi kebugaran dirancang sangat *OOS-heavy* dan memprioritaskan profil risiko:

$$\begin{aligned} \text{Score} = & (0.40 * \text{avg_oos_profit_factor}) \\ & + (0.25 * \text{oos_regime_stability}) \\ & - (0.20 * \text{risk_drawdown_penalty}) \\ & + (0.15 * \text{is_average_performance}) \end{aligned}$$

Komponen Risk Metric:

- `risk_drawdown_penalty`: Menghukum strategi berdasarkan **Ulcer Index** atau standar deviasi dari rentetan kekalahan (*Consecutive Losses*). Strategi dengan kurva ekuitas bergerigi (butuh waktu lama untuk *recovery* dari *Drawdown*) akan mendapatkan skor jauh lebih rendah daripada kurva ekuitas yang mulus.

9. Pipeline Eksekusi & Top-N Strategy Selection

Alur kerja asinkron berkelanjutan (berbasis *tokio channels*) dirancang untuk beroperasi secara mandiri sekaligus menyajikan metrik visual:

A. Autonomous Orchestration Cycle

1. **Auto-Start Telemetry:** Menghidupkan *web-server* Dashboard (*Dioxus/Axum*) di latar belakang secara konkuren untuk pemantauan *live*.
2. **Generation:** Mensintesis AST awal (Primitif + Terminal mikrostruktur).
3. **Tuning (Bayesian):** Mengevaluasi rentang parameter di sampel *In-Sample* awal.
4. **Vectorized Backtest:** Menguji memori simulasi Rust (*chunking*) melintasi matriks *Walk-Forward Multi-Window*.
5. **Noise & Regime Check:** Menguji daya tahan terhadap *slippage/spread injection* dan menghitung *Regime Survival*.

6. **Database Persistence:** Mendorong hasil transaksi dan mutasi *fitness* ke PostgreSQL (sqlx).

B. Top-N Strategy Selection Engine

- **Ranking:** Populasi diurutkan berdasarkan *Fitness Score* final.
- **Top 50 Selection:** Mesin secara otomatis menyaring dan mengunci mutlak **50 strategi teratas**.
- **Strategy Export Layer:** Mengeluarkan "*Golden Strategy Set*" ini ke dalam bentuk JSON (memuat AST Enums, params_json, OOS Metrics, dan Regime Profile) agar dapat dieksekusi di *trading engine* eksternal.

10. Observability & Auto-Running Dashboard

Sistem mensyaratkan bahwa setiap kali pencarian (Evolutionary Engine) dihidupkan, **Full Research Dashboard akan otomatis menyala dan berjalan secara konkuren**. *Dashboard* ini dirender sebagai tokio task terpisah sehingga nol-gesekan (*zero friction*) terhadap kecepatan komputasi *Engine* utama.

A. Evolutionary Overview (Macro View)

- **Live Fitness Convergence:** Kurva interaktif *real-time* yang menunjukkan pergerakan rata-rata skor populasi di setiap generasi, mendeteksi jika mesin mentok (*plateau*).
- **Elimination Funnel:** Diagram yang melacak rasio kematian strategi secara instan (Contoh: 10,000 Generated -> 1,500 Failed Hard Filters -> 450 Failed OOS -> 50 Golden Set).
- **Diversity Heatmap:** Representasi visual tingkat kemiripan genetik. Berwarna merah jika populasi menderita "Insens Genetik", hijau jika sehat.

B. Strategy Deep-Dive (Micro View)

- **Interactive AST Tree:** Representasi grafis pohon genetik (node dan cabang) untuk membedah anatomi "*Alpha*" dari setiap strategi yang lolos.
- **Split Equity Curve (IS vs OOS):** Grafik kurva keseimbangan modal yang memisahkan *In-Sample* (biru) dan *Out-Of-Sample* (hijau).
- **Trade Scatter & Underwater Chart:** Visualisasi titik-titik BUY/SELL di atas *candlestick* harga, beserta kedalaman dan durasi *Drawdown*.

C. Feature Importance & Explainability (XAI)

Karena *Genetic Programming* sering dianggap sebagai "kotak hitam", dashboard dilengkapi dengan analitik XAI untuk menerjemahkan logika mesin agar bisa dipahami oleh analis:

- **Global Feature Dominance:** Grafik interaktif yang melacak fitur mikrostruktur mana yang mendominasi. Memberikan wawasan kritis seperti: "*Dari 50 strategi teratas, fitur Tick Density muncul di 80% pohon strategi, mengindikasikan Alpha terbesar bulan ini berasal dari anomali aktivitas tick.*"
- **Operator & Node Heatmap:** Visualisasi matriks yang menyoroti operator matematika

mana (sigmoid, add, div) yang paling sering berpasangan dengan fitur tertentu, membedah secara pasti *bagaimana* algoritma pasar dieksploitasi oleh sistem.

D. Regime Analytics & Diagnostics

- **Market Regime Heatmap:** Tabel historis yang dikodekan warna berdasarkan profil *Hurst Exponent* & Volatilitas di setiap *Window*.
- **Cross-Regime Yield Matrix:** Melacak performa *Win-Rate* strategi saat melintasi berbagai rezim untuk memverifikasi ketahanannya di pasar *Ranging* maupun *Trending*.

E. System Telemetry

- **Hardware & Engine Speedometer:** Memantau serapan RAM (*Chunking Status*), beban *thread* rayon, utilisasi VRAM GPU, serta latensi *Rows Processed per Second* (RPS) secara *live*.

11. Rekomendasi Tech Stack Terpadu (Research Layer)

Komponen Sistem	Teknologi Utama (Rust Ecosystem)	Fungsi / Keunggulan
Dataframe / Analytics	polars (Rust Native API)	Akses memori langsung & vektorisasi super cepat untuk analitik data.
Genetic Programming	Custom AST (Enums) + rayon	Kecepatan evaluasi <i>pattern matching</i> Rust dan paralelisasi CPU.
Parameter Tuning	PyO3 (FFI) + Optuna	Memanggil heuristik Bayesian canggih (Python) langsung dari dalam aplikasi Rust.
Hardware Acceleration	cudarc (CUDA) / wgpu	Akses VRAM (GPU) langsung untuk komputasi matematika SIMT massal.
Backtesting Engine	barter-rs / Custom tokio	<i>Engine event-driven</i> untuk simulasi mikrostruktur.
Data Loading	clickhouse.rs	Kueri asinkron <i>columnar</i> ter- <i>chunk</i> untuk memompa data ke RAM secara <i>gapless</i> .

Research DB Connect	sqlx	Integrasi PostgreSQL yang divalidasi tipe (<i>Type-safe</i>) untuk penyimpanan DNA.
Auto-Running Dashboard	dioxus / axum (Concurrent)	Menjalankan <i>Web Server / UI</i> bersamaan di <i>thread</i> asinkron terpisah tanpa <i>blocking</i> .

Dokumen spesifikasi teknis ESSE v1.0 - Core Discovery Engine (Architecturally Robust).